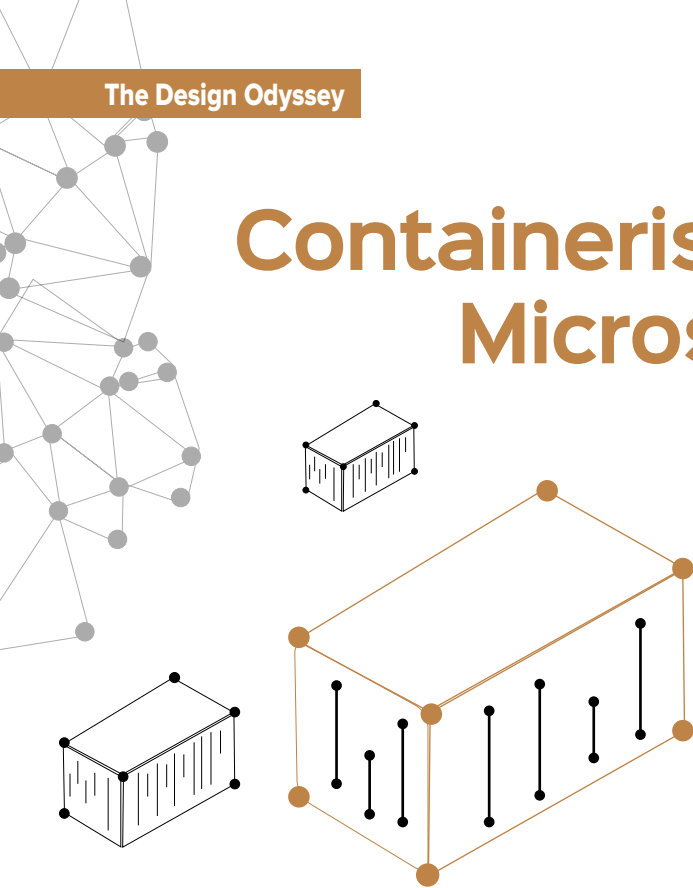


Containerisation of Java Microservices



Frequent roll outs of microservices demand self-contained units that ensure safe and successful deployments. Containerisation offers an elegant solution. In this part 7 of the Design Odyssey, we will build and deploy the AddService of UMS as a Docker container.

In the previous part of this series, it was observed that a packaged Spring Boot application carries an embedded Tomcat Web server along with class files and dependent libraries. However, the JRE was left out of the application package. The deployment engineer is still required to find and install a suitable JRE before deploying the application. How about including the JRE also into the package and distributing it so that the deployment can be automated fully?

That's exactly what containerisation is!

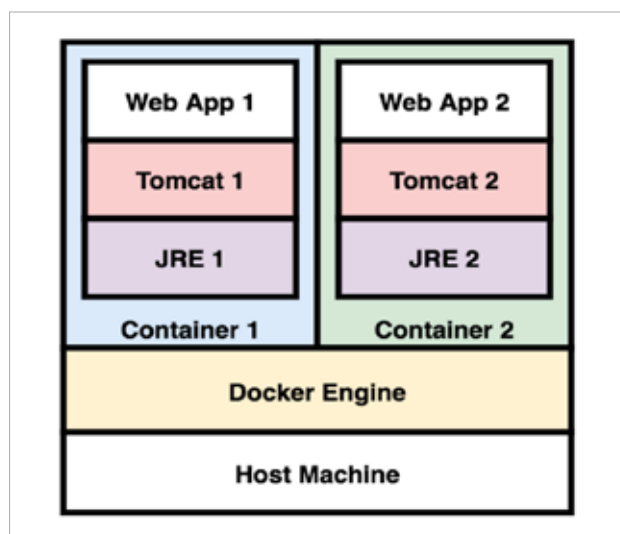


Figure 1: Deployment unit with containerisation

As depicted in Figure 1, a container is a deployment unit that consists of not only the application code but also the required operating environment, which includes artifacts like JRE, Python runtime, Node runtime, etc, as required.

Docker containers

In theory, the concept of containerisation is not very new. For example, the Linux operating system has already been offering the necessary bits and pieces in that direction. However, credit goes to the Docker containers for the popularity of this approach in the last decade.

The architecture of Docker containers can be summarised as shown in Figure 2. It consists of: (1) a Docker Hub, which is a repository of distributable images of Docker containers,

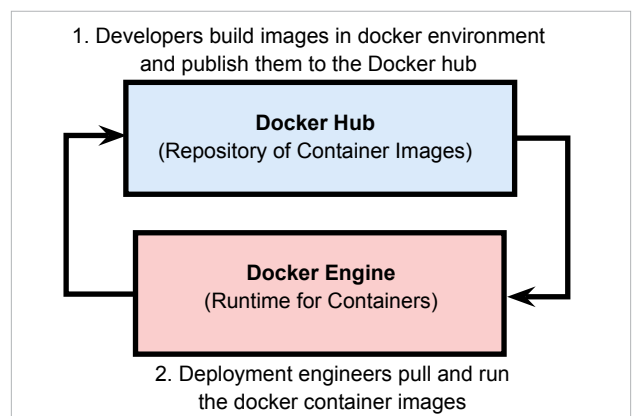


Figure 2: The Docker architecture